

D-LAMP:

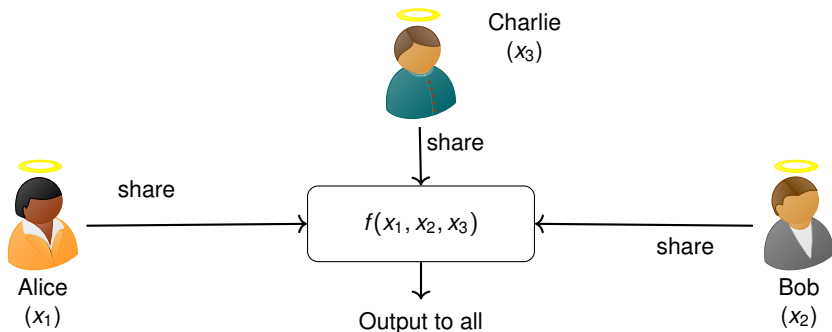
Discrete LAPlacian MPC Noise-Generating
Protocols for Differentially Private Computation

Sebastian David Albu

April 22 2025

Definitions and Background

Secure Multiparty Computation (MPC)



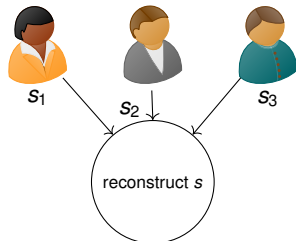
MPC lets n parties jointly compute a function $f(x_1, \dots, x_n)$ over their private inputs, revealing only the output and nothing else [EKR+18].

Secret Sharing: Arithmetic vs. Binary Shares

Secret sharing splits a secret s into shares $\{s_i\}$ so that no subset below threshold learns s , but authorized subsets can reconstruct [EKR+18].

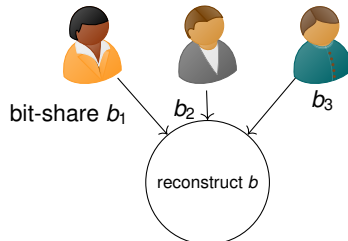
Arithmetic shares $\langle r \rangle \pmod{p}$:

$$s = \sum_{i=1}^n s_i \pmod{p}$$

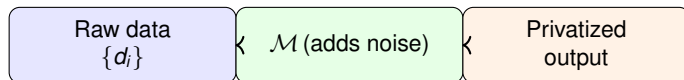


Binary (XOR) shares $[r]$:

$$s = \bigoplus_{i=1}^n s_i$$



Differential Privacy (DP)



A randomized mechanism $\mathcal{M}$ satisfies ϵ -DP if for any two neighboring datasets D, D' , and any output o ,

$$\Pr[\mathcal{M}(D) = o] \leq e^\epsilon \Pr[\mathcal{M}(D') = o].$$

This ensures that the presence or absence of any single record has only a bounded effect on the output [DKM+06].

In practice, DP provides a way to analyze data without compromising individual privacy, even when data is publicly released or shared with third parties.

Motivation

Combining MPC & DP

- In today's data-driven world many organizations (banks, hospitals) need to compute over sensitive data without pooling it centrally.
- MPC offers exact joint computation but exact outputs can leak private information.
- Differential privacy mitigates output leakage by adding noise, but doing so in a fully distributed setting introduces new obstacles:

Challenges in Combining MPC & DP

- **Distributed non-uniform sampling:** Parties must jointly sample from, e.g., a discrete Laplace distribution, even if some are malicious.
- **Adversarial randomness:** If each party independently picks noise r_i , a corrupt party can bias or cancel out the total noise.
- **No trusted curator:** Unlike standalone DP, there is no single “mechanism” node, noise generation itself must be secure MPC.
- **Share-type conversions:** Switching between arithmetic shares (for addition) and binary shares (for comparisons) incurs heavy communication.

Motivation: Privacy-Preserving Noise Generation

- **Our gap:** Existing MPC-DP protocols either incur high communication or assume a semi-honest setting.
- **Our goal:** Use an existing, active-secure noise-generation protocol (FDL1) [EIKN22] that uses a discrete Laplace mechanism with low rounds and bandwidth and optimize it further.

The FDL1 Protocol

Key Components of FDL1 Protocol [EIKN22]

- **Discrete Laplace Sampling:** Parties jointly sample shares of a discrete Laplace random variable without revealing each individual share [GRS08].
- **Mixed-Circuit Design:** Arithmetic for share-addition, binary for sign and magnitude; we interleave them to minimize expensive conversions. (Similar to other FDL from [EIKN22; MRS24])
- **Domain-Shift Mechanism:** Convert arithmetic shares into bitwise shares using pre-distributed RR pairs, avoiding heavy CRT-based routines.

FDL1:

Input: $([a_{i1}], \dots, [a_{id}])_{i \in [0..c)}, ([b_{i1}], \dots, [b_{id}])_{i \in [0..c)}$, where $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1 \left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)} \right) \right]$.

Binary Domain:

- 1) For each $i \in [0..c)$, compute $[a_i] = \text{BER}_{\beta_i, d} \left([a_{ij}]_{j \in [d]} \right)$.
- 2) For each $i \in [0..c)$, compute $[b_i] = \text{BER}_{\beta_i, d} \left([b_{ij}]_{j \in [d]} \right)$.

Arithmetic Domain:

- 3) Compute $[x_1] = \text{GEO}_{p, N} \left([a_i]_{i \in [0..c)} \right)$.
- 4) Compute $[x_2] = \text{GEO}_{p, N} \left([b_i]_{i \in [0..c)} \right)$.

FDL1:

Input: $([a_{i1}], \dots, [a_{id}])_{i \in [0..c)}, ([b_{i1}], \dots, [b_{id}])_{i \in [0..c)}$, where $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1 \left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)} \right) \right]$.

Binary Domain:

- 1) For each $i \in [0..c)$, compute $[a_i] = \text{BER}_{\beta_i, d} \left([a_{ij}]_{j \in [d]} \right)$.
- 2) For each $i \in [0..c)$, compute $[b_i] = \text{BER}_{\beta_i, d} \left([b_{ij}]_{j \in [d]} \right)$.

Arithmetic Domain:

- 3) Compute $[x_1] = \text{GEO}_{p, N} \left([a_i]_{i \in [0..c)} \right)$.
- 4) Compute $[x_2] = \text{GEO}_{p, N} \left([b_i]_{i \in [0..c)} \right)$.

Input of Procedure

We assume, as input, a collection

$$\left([a_{i1}], \dots, [a_{id}] \right)_{i \in [0..c)} \quad \text{and} \quad \left([b_{i1}], \dots, [b_{id}] \right)_{i \in [0..c)},$$

where $a_{ij}, b_{ij} \in \{0, 1\}$.

- These represent c vectors of d independent simulated coin flips generated by a seeded PRNG using the MPSDZ [MP-25] framework.
- Let $c, N, M \in \mathbb{N}$, whereas it holds that $N = 2^c$, $M < N$, and $\beta_i = (1 + p^{2^{-i}})^{-1}$ for $i \in [0..c)$.
- For the procedure we use SPDZ2K secret shares [CDE+18].

FDL1:

Input: $([a_{i1}], \dots, [a_{id}])_{i \in [0..c)}, ([b_{i1}], \dots, [b_{id}])_{i \in [0..c)}$, where $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1 \left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)} \right) \right]$.

Binary Domain:

- 1) For each $i \in [0..c)$, compute $[a_i] = \text{BER}_{\beta_i, d} \left([a_{ij}]_{j \in [d]} \right)$.
- 2) For each $i \in [0..c)$, compute $[b_i] = \text{BER}_{\beta_i, d} \left([b_{ij}]_{j \in [d]} \right)$.

Arithmetic Domain:

- 3) Compute $[x_1] = \text{GEO}_{p, N} \left([a_i]_{i \in [0..c)} \right)$.
- 4) Compute $[x_2] = \text{GEO}_{p, N} \left([b_i]_{i \in [0..c)} \right)$.

Approximate Bernoulli Procedure (BER) in Binary Domain

Input & Output

- **Input:** n_{parties} , N , d_{prob} , $\alpha = (\alpha_1, \dots, \alpha_{d_{\text{prob}}})$ in LSB-first form.
- **Output:** Binary shares $[b_1], \dots, [b_N]$ with $b_i \in \{0, 1\}$.

Approximate Bernoulli Procedure (BER) in Binary Domain

Protocol Steps

1. For each sample $i = 0, \dots, N - 1$, each party uses its seeded PRNG to generate d_{prob} bits $r_{i \cdot d_{\text{prob}} + 1}, \dots, r_{(i+1) \cdot d_{\text{prob}}}$.
2. Collect into $r^{(i)}$.
3. Compare $r^{(i)}$ with α via an unsigned less-than circuit:

$$[b_i] = \text{binary_less_than_unsigned}(r^{(i)}, \alpha).$$

4. Output $[b_1], \dots, [b_N]$.

Subroutine: Unsigned Less–Than Circuit

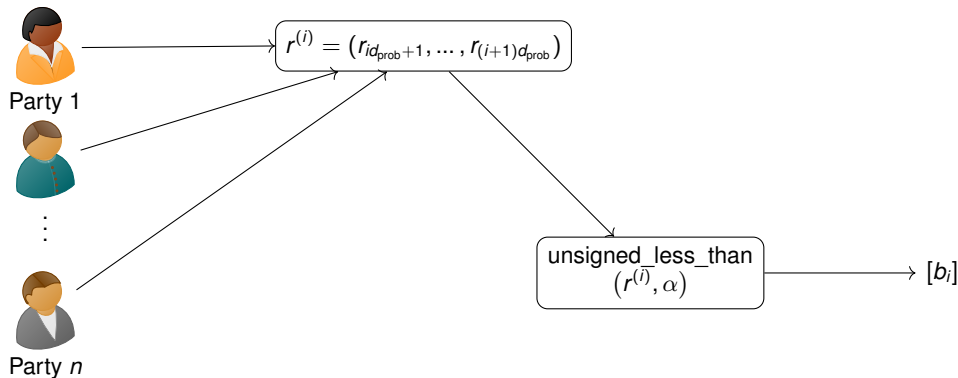
Purpose

Compare two d -bit vectors $a = (a_0, \dots, a_{d-1})$ and $b = (b_0, \dots, b_{d-1})$ (LSB–first) and output secret bit $r = 1$ iff $a < b$.

Inputs & Output

- $a, b \in \{0, 1\}^d$ (LSB–first).
- Output $r = \text{borrow} \in \{0, 1\}$.

Approximate Bernoulli Procedure (BER) in Binary Domain



FDL1:

Input: $([a_{i1}], \dots, [a_{id}])_{i \in [0..c)}, ([b_{i1}], \dots, [b_{id}])_{i \in [0..c)}$, where $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1 \left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)} \right) \right]$.

Binary Domain:

- 1) For each $i \in [0..c)$, compute $[a_i] = \text{BER}_{\beta_i, d} \left([a_{ij}]_{j \in [d]} \right)$.
- 2) For each $i \in [0..c)$, compute $[b_i] = \text{BER}_{\beta_i, d} \left([b_{ij}]_{j \in [d]} \right)$.

Arithmetic Domain:

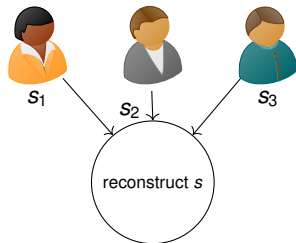
- 3) Compute $[x_1] = \text{GEO}_{\rho, N} \left([a_i]_{i \in [0..c)} \right)$.
- 4) Compute $[x_2] = \text{GEO}_{\rho, N} \left([b_i]_{i \in [0..c)} \right)$.

Secret Sharing: Arithmetic vs. Binary Shares

Remember there are two domains here:

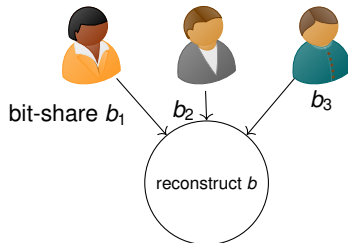
Arithmetic shares $\langle r \rangle \pmod{p}$:

$$s = \sum_{i=1}^n s_i \pmod{p}$$



Binary (XOR) shares $[r]$:

$$s = \bigoplus_{i=1}^n s_i$$



Domain Transformation: Binary $\rightarrow$ Arithmetic via daBits

Goal

Convert secret-shared bits $[b_i] \in \{0, 1\} \pmod{2}$ into arithmetic shares $\langle b_i \rangle \pmod{p}$ or 2^k .

1. **Prepare daBits:** Offline, generate pairs $(\langle r_i \rangle, [r_i])$ for random bits r_i .
2. **Mask:** Locally compute $[b_i \oplus r_i]$ and open it to obtain public $m_i = b_i \oplus r_i$.
3. **Adjust arithmetic share:**

$$\langle b_i \rangle = \langle r_i \rangle - 2 m_i \langle r_i \rangle + m_i.$$

4. **Output:** The list $(\langle b_i \rangle)_{i=0}^{m-1}$.

Geometric Distribution Procedure (GEO)

Input & Output

■ **Input:** Binary shares $([b_0], \dots, [b_{d-1}])$ in LSB–MSB order.

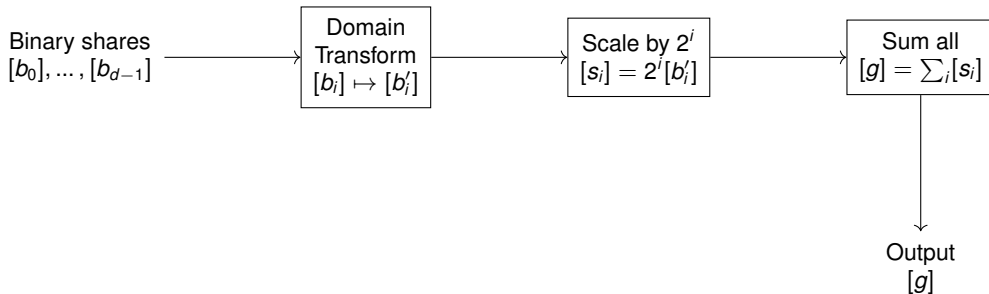
■ **Output:** Arithmetic share $[g] = \sum_{i=0}^{d-1} 2^i b_i$.

Geometric Distribution Procedure (GEO)

Procedure Steps

1. **Domain transformation:** Convert each $[b_i]$ to arithmetic share $[b'_i]$.
2. **Scaling:** Compute $[s_i] = 2^i \cdot [b'_i]$ for $i = 0, \dots, d - 1$.
3. **Summation:** Compute $[g] = \sum_{i=0}^{d-1} [s_i]$.

Geometric Distribution Procedure (GEO)



FDL1

Arithmetic Domain:

- 5) Compute the difference $[y] = [x_1] - [x_2]$.

Binary Domain:

- 6) If $|y| \leq M$, then output $[y]$; otherwise, output $\perp$.

Arithmetic Domain:

(The result $[y]$ needs to be transformed to be in the arithmetic domain)

FDL1

Arithmetic Domain:

5) Compute the difference $[y] = [x_1] - [x_2]$.

Binary Domain:

6) If $|y| \leq M$, then output $[y]$; otherwise, output $\perp$.

Arithmetic Domain:

(The result $[y]$ needs to be transformed to be in the arithmetic domain)

- The difference $[y] = [x_1] - [x_2]$ is trivial in the arithmetic domain

Drawbacks of original FDL1 protocol

- The GEO procedure causes a domain shift.
- After step 5 we already need to convert back to binary domain.
- Getting absolute value $|y|$ is not possible in arithmetic domain.
- A comparison circuit is also not possible in arithmetic domain.

Subroutine: Arithmetic $\rightarrow$ Binary via daBits

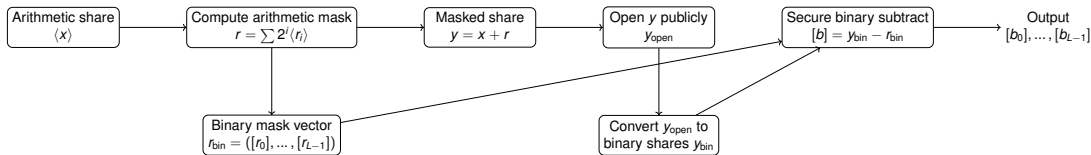
Input & Output

- **Input:** Arithmetic share $\langle x \rangle \in \mathbb{Z}_{2^k}$, bit-length L , and L daBit pairs $(\langle r_i \rangle, [r_i])$.
- **Output:** Binary shares $[b_0], \dots, [b_{L-1}]$ reconstructing x in two's-complement.

Subroutine: Arithmetic $\rightarrow$ Binary via daBits

1. Ensure L daBits $\{(\langle r_i \rangle, [r_i])\}_{i=0}^{L-1}$ are available.
2. Compute arithmetic mask $r = \sum_{i=0}^{L-1} 2^i \langle r_i \rangle$ and binary mask vector $r_{\text{bin}} = ([r_0], \dots, [r_{L-1}])$.
3. Mask input: $y = x + r$ (in the arithmetic domain).
4. Open y to obtain public y_{open} .
5. Convert y_{open} into binary shares $y_{\text{bin}} = ([y_0], \dots, [y_{L-1}])$.
6. Securely compute $[b_0, \dots, b_{L-1}] = y_{\text{bin}} - r_{\text{bin}}$ via a binary subtraction circuit.
7. Output $[b_0], \dots, [b_{L-1}]$.

Arithmetic $\rightarrow$ Binary Conversion via daBits



Observations

- The arithmetic to binary sub-routine is expensive and slow
- We only shift the domain for a subtraction
- Intuitively, the value of the binary share is the same as the arithmetic share
- So, GEO does essentially *only* transform the computation domain!
- We are able to subtract also in binary domain
- We already *need* this subtraction circuit for the arithmetic to binary conversion!
- Why not just stay completely in the binary domain?

Optimizing the Protocol: FDL1 Only in the Binary Domain

FDL1 Only in Binary Domain

Input: $\left([a_{i1}], \dots, [a_{id}]\right)_{i \in [0..c)}$ and $\left([b_{i1}], \dots, [b_{id}]\right)_{i \in [0..c)}$,
with $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1\left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)}\right)\right]$.

Binary Domain:

- 1) For each $i \in [0..c)$, compute $[a_i] = \text{BER}_{\beta_i, d}\left([a_{ij}]_{j \in [d]}\right)$.
- 2) For each $i \in [0..c)$, compute $[b_i] = \text{BER}_{\beta_i, d}\left([b_{ij}]_{j \in [d]}\right)$.
- 3) Compute the difference $[y] = [a_1] - [a_2]$.
- 4) If $|y| \leq M$, then output $[y]$; otherwise, output $\perp$.

Arithmetic Domain:

(The result $[y]$ needs to be transformed to be in the arithmetic domain)

FDL1 Only in Binary Domain

Input: $\left([a_{i1}], \dots, [a_{id}]\right)_{i \in [0..c]}$ and $\left([b_{i1}], \dots, [b_{id}]\right)_{i \in [0..c]}$,
with $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1\left(\left(a_{i1}, \dots, a_{id}\right)_{i \in [0..c]}, \left(b_{i1}, \dots, b_{id}\right)_{i \in [0..c]}\right)\right]$.

Binary Domain:

- 1) For each $i \in [0..c]$, compute $[a_i] = \text{BER}_{\beta_i, d}\left([a_{ij}]_{j \in [d]}\right)$.
- 2) For each $i \in [0..c]$, compute $[b_i] = \text{BER}_{\beta_i, d}\left([b_{ij}]_{j \in [d]}\right)$.
- 3) Compute the difference $[y] = [a_1] - [a_2]$.
- 4) If $|y| \leq M$, then output $[y]$; otherwise, output $\perp$.

Arithmetic Domain:

(The result $[y]$ needs to be transformed to be in the arithmetic domain)

FDL1 in Binary Domain:

- The input is the same as in the previous version
- The BER procedure is also identical
- We now skip the GEO procedure and directly subtract $[a_i] - [b_i]$

FDL1 Only in Binary Domain

Input: $\left([a_{i1}], \dots, [a_{id}]\right)_{i \in [0..c)}$ and $\left([b_{i1}], \dots, [b_{id}]\right)_{i \in [0..c)}$,
with $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1\left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)}\right)\right]$.

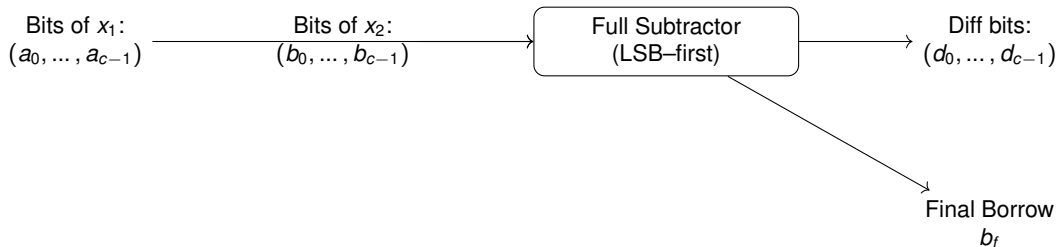
Binary Domain:

- 1) For each $i \in [0..c)$, compute $[a_i] = \text{BER}_{\beta_i, d}\left([a_{ij}]_{j \in [d]}\right)$.
- 2) For each $i \in [0..c)$, compute $[b_i] = \text{BER}_{\beta_i, d}\left([b_{ij}]_{j \in [d]}\right)$.
- 3) Compute the difference $[y] = [a_1] - [a_2]$.
- 4) If $|y| \leq M$, then output $[y]$; otherwise, output $\perp$.

Arithmetic Domain:

(The result $[y]$ needs to be transformed to be in the arithmetic domain)

Binary Subtraction with Final Borrow



Fixed-Length Extension to Two's-Complement

Input & Output

- **Input:** c -bit diff $(d_0, \dots, d_{c-1})$ and final borrow b_f .
- **Output:** L -bit two's-complement vector ($L = \text{SPDZ2K_SIZE}$). (Set to 64 in our project)

1. For $i = 0, \dots, c - 1$: set bit i to d_i .
2. For $i = c, \dots, L - 1$: set bit i to b_f (the sign extension).
3. Output the L -bit vector.

Binary Subtraction in Two's-Complement

End-to-End Procedure

1. Compute unsigned subtraction of $x_1 - x_2$ (LSB-first) to get $(d_0, \dots, d_{c-1})$ and final borrow b_f .
2. Extend to L bits by sign-extending the borrow:

$$(d_0, \dots, d_{c-1}, \underbrace{b_f, \dots, b_f}_{L-c \text{ times}}).$$

3. Result is the two's-complement representation of $x_1 - x_2$ as an L -bit secret vector.

FDL1 Only in Binary Domain

Input: $\left([a_{i1}], \dots, [a_{id}]\right)_{i \in [0..c)}$ and $\left([b_{i1}], \dots, [b_{id}]\right)_{i \in [0..c)}$,
with $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1\left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)}\right)\right]$.

Binary Domain:

- 1) For each $i \in [0..c)$, compute $[a_i] = \text{BER}_{\beta_i, d}\left([a_{ij}]_{j \in [d]}\right)$.
- 2) For each $i \in [0..c)$, compute $[b_i] = \text{BER}_{\beta_i, d}\left([b_{ij}]_{j \in [d]}\right)$.
- 3) Compute the difference $[y] = [a_1] - [a_2]$.
- 4) If $|y| \leq M$, then output $[y]$; otherwise, output $\perp$.

Arithmetic Domain:

(The result $[y]$ needs to be transformed to be in the arithmetic domain)

Secure $\geq$ from Binary Vectors

Input & Output

Input: Two secret binary vectors $A, B \in \{0, 1\}^d$ (LSB-first). **Output:** Secret bit r with

$$r = \begin{cases} 1, & A \geq B, \\ 0, & \text{otherwise.} \end{cases}$$

Procedure

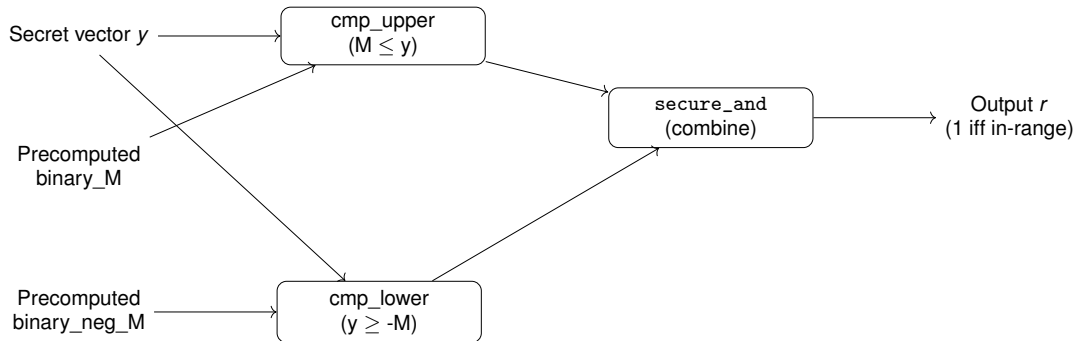
1. Compute $\text{diff} = A - B$ via the two's-complement subtraction circuit.
2. Let sign be the MSB of diff (last bit in LSB-first).
3. Return $r = \text{secure_not}(\text{sign})$.

Optimizing the Protocol

Secure $\geq$ from Binary Vectors



Secure In-Range Check – $-M \leq y \leq M$



Results and Comparisons

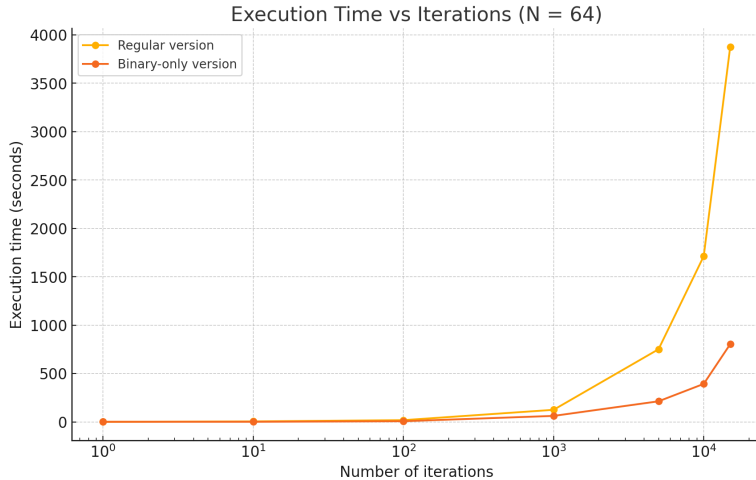
Results

Comparison for number of samples

- Always run with $p = 0.5$ and security parameter $N = 64$
- Always 2 parties
- Online data transmission is per party only
- Examined iterations:
 - 1
 - 10
 - 100
 - 1000
 - 5000
 - 10000
 - 15000

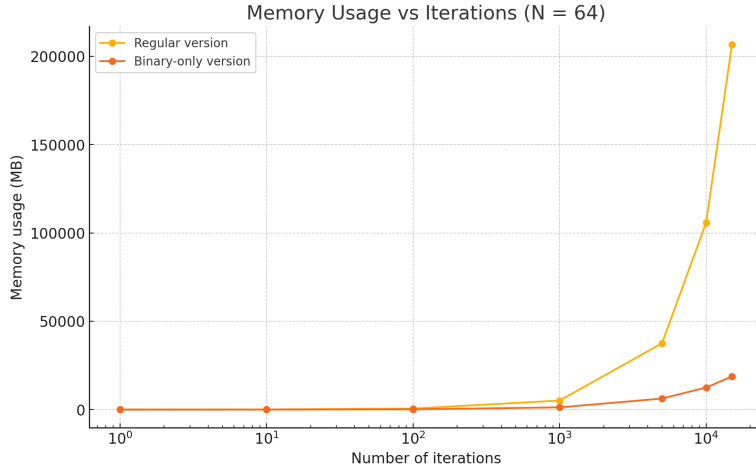
Results

Computation Time



Results

Memory Usage



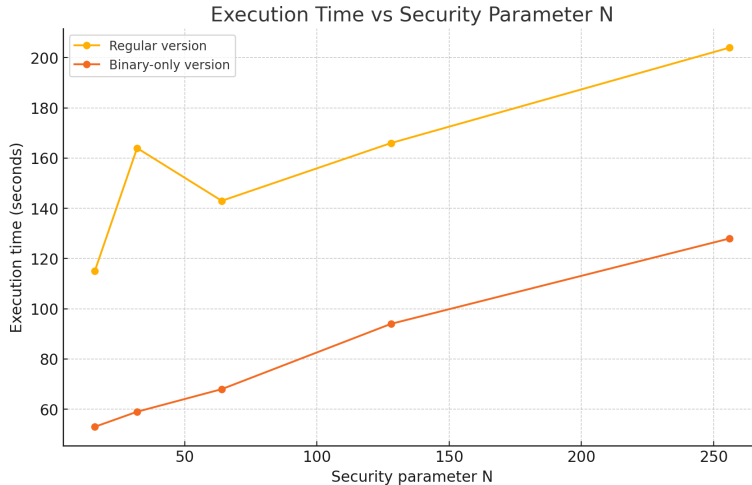
Results

Comparison for security parameter N

- Always run with $p = 0.5$ and 1000 iterations
- Always 2 parties
- Online data transmission is per party only
- Examined parameters:
 - 16
 - 32
 - 64
 - 128
 - 256

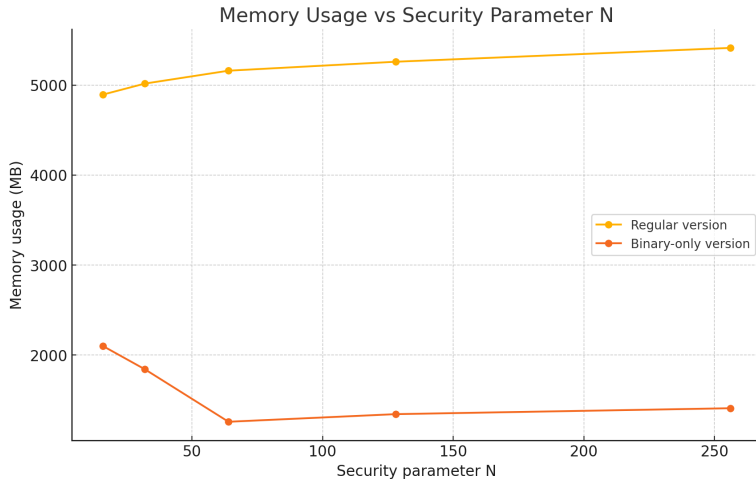
Results

Computation Time



Results

Memory Usage



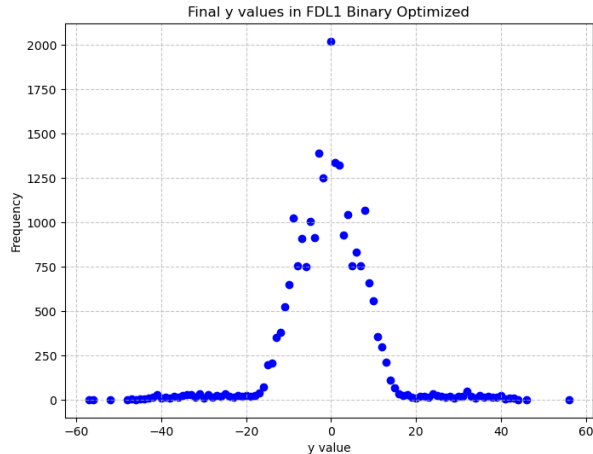
Results

Plots showing distribution

- Always run with $p = 0.5$, and 1000 iterations
- Always used Binary-Only version
- (Regular version produces same plots, but inefficient)
- Always 2 parties
- For simplicity $M = N - 1$
- Examined security parameters:
 - $N = 64$ with 15000 iterations
 - $N = 128$ with 10000 iterations

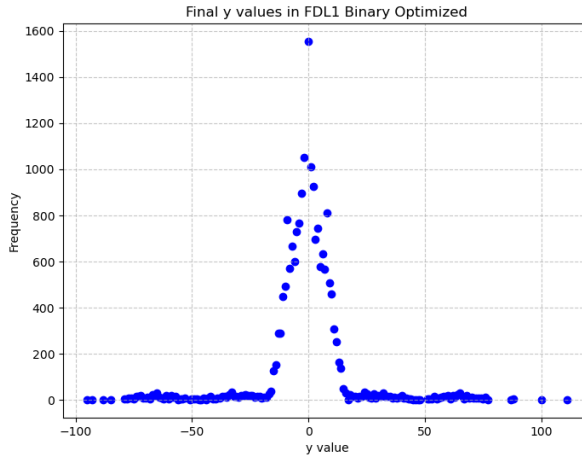
Results

FDL1 $N = 64$ with 15000 iterations



Results

FDL1 $N = 128$ with 10000 iterations



Questions?

References I

2

- [CDE+18] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. **SPDZ^K: Efficient MPC mod 2^k for Dishonest Majority**. Annual International Cryptology Conference. Springer. 2018, pp. 769–798.

References II

- [DKM+06] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. **Our data, ourselves: Privacy via distributed noise generation.** Advances in cryptology-EUROCRYPT 2006: 24th annual international conference on the theory and applications of cryptographic techniques, st. Petersburg, Russia, May 28-June 1, 2006. proceedings 25. Springer. 2006, pp. 486–503.
- [EIKN22] Reo Eriguchi, Atsunori Ichikawa, Noboru Kunihiro, and Koji Nuida. **Efficient Noise Generation Protocols for Differentially Private Multiparty Computation.** Cryptology ePrint Archive, Paper 2022/1391. 2022. URL: <https://eprint.iacr.org/2022/1391>.

References III

- [EKR+18] David Evans, Vladimir Kolesnikov, Mike Rosulek, et al. **A pragmatic introduction to secure multi-party computation.** *Foundations and Trends® in Privacy and Security* 2.2-3 (2018), pp. 70–246.
- [GRS08] Arpita Ghosh, Tim Roughgarden, and Mukund Sundararajan. **Universally Utility-Maximizing Privacy Mechanisms.** *CoRR* abs/0811.2841 (2008). arXiv: 0811.2841. URL: <http://arxiv.org/abs/0811.2841>.
- [MP-25] MP-SPDZ. **MP-SPDZ: A Framework for Secure Multiparty Computation.** <https://mp-spdz.readthedocs.io/en/latest/>. Accessed: 2025-04-11. 2025.

References IV

- [MRS24] Fredrik Meisingseth, Christian Rechberger, and Fabian Schmid. **Practical Two-party Computational Differential Privacy with Active Security.** Cryptology ePrint Archive, Paper 2024/004. 2024. URL: <https://eprint.iacr.org/2024/004>.

D-LAMP:

Discrete LAPlacian MPC Noise-Generating
Protocols for Differentially Private Computation

Sebastian David Albu

April 22 2025